

OSSP SKILL PROGRAMS

1) Set up process

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid;

    printf("OSSP Program 01\n");

    pid = fork();

    if(pid < 0)
    {
        printf("Fork failed\n");
        return 1;
    }

    else if(pid == 0)
    {
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());

        execl("/bin/ls", "ls", NULL);

        printf("Exec failed\n");
    }

    else
    {
        printf("Parent Process\n");
        printf("Parent PID: %d\n", getpid());

        waitpid(pid, NULL, 0);

        printf("Child completed\n");
    }
}
```

```
balaji_p@Harshi:~$ mkdir -p ~/2520030104_OSSP
balaji_p@Harshi:~$ cd ~/2520030104_OSSP
balaji_p@Harshi:~/2520030104_OSSP$ nano program1.c
balaji_p@Harshi:~/2520030104_OSSP$ gcc program1.c -o program1
balaji_p@Harshi:~/2520030104_OSSP$ ./program1
OSSP Program 01
Parent Process
Child Process
Parent PID: 559
Child PID: 560
program1 program1.c
Child completed
balaji_p@Harshi:~/2520030104_OSSP$ nano program1.c
balaji_p@Harshi:~/2520030104_OSSP$
```

2) Shell loop

```

#include <stdio.h>
#include <string.h>

int main()
{
    char command[100];

    while(1)
    {
        printf("OSSP-Shell> ");

        fgets(command, sizeof(command), stdin);

        command[strlen(command)-1] = '\0';

        if(strcmp(command, "exit") == 0)
        {
            printf("Shell terminated\n");
            break;
        }

        printf("You entered: %s\n", command);
    }

    return 0;
}

```

```

balaji_p@Harshi:~/2520030104_OSSP$ nano shell.c
balaji_p@Harshi:~/2520030104_OSSP$ gcc shell.c -o shell
balaji_p@Harshi:~/2520030104_OSSP$ ./shell
OSSP-Shell> hello
You entered: hello
OSSP-Shell> ls
You entered: ls
OSSP-Shell> pwd
You entered: pwd
OSSP-Shell> exit
Shell terminated
balaji_p@Harshi:~/2520030104_OSSP$

```

3) Command history

```

#include <stdio.h>
#include <string.h>

int main()
{
    char history[10][100];
    char command[100];
    int count = 0;

    while(1)
    {
        printf("OSSP-Shell> ");

        fgets(command, 100, stdin);

        command[strlen(command)-1] = '\0';

        if(strcmp(command, "exit") == 0)
            break;

        if(strcmp(command, "history") == 0)
        {
            for(int i = 0; i < count; i++)
            {
                printf("%d %s\n", i + 1, history[i]);
            }

            continue;
        }

        strcpy(history[count], command);
        count++;
    }

    return 0;
}

```

```

balaji_p@Harshi:~/2520030104_OSSP$ nano history.c
balaji_p@Harshi:~/2520030104_OSSP$ gcc history.c -o history
balaji_p@Harshi:~/2520030104_OSSP$ ./history
OSSP-Shell> ls
OSSP-Shell> pwd
OSSP-Shell> hello
OSSP-Shell> history
1 ls
2 pwd
3 hello
OSSP-Shell>

```

4) Command parser

```
#include <stdio.h>
#include <string.h>

int main()
{
    char input[200];
    char *token;

    printf("Enter command: ");

    fgets(input, 200, stdin);

    token = strtok(input, " ");

    while(token != NULL)
    {
        printf("Token: %s\n", token);

        token = strtok(NULL, " ");
    }

    return 0;
}
```

```
balaji_p@Harshi:~/2520030104_OSSP$ nano tokenize.c
balaji_p@Harshi:~/2520030104_OSSP$ gcc tokenize.c -o tokenize
balaji_p@Harshi:~/2520030104_OSSP$ ./tokenize
Enter command: ls -l /home
Token: ls
Token: -l
Token: /home
balaji_p@Harshi:~/2520030104_OSSP$
```

5) Quote handler

```
#include <stdio.h>
#include <string.h>

int main()
{
    char input[200];

    printf("Enter command: ");

    fgets(input, 200, stdin);

    if(input[0] == '\\')
    {
        printf("Quoted string detected\n");
    }

    printf("Command: %s", input);

    return 0;
}
```

```
balaji_p@Harshi:~/2520030104_OSSP$ nano quote.c
balaji_p@Harshi:~/2520030104_OSSP$ gcc quote.c -o quote
balaji_p@Harshi:~/2520030104_OSSP$ ./quote
Enter command: hello world
Command: hello world
```

6) Escape handler

```

#include <stdio.h>
#include <string.h>

void remove_escape(char *input)
{
    int i, j = 0;

    for(i = 0; input[i] != '\0'; i++)
    {
        if(input[i] == '\\')
        {
            continue;
        }

        input[j++] = input[i];
    }

    input[j] = '\0';
}

int main()
{
    char input[200];

    printf("Enter command: ");

    fgets(input, 200, stdin);

    input[strlen(input) - 1] = '\0';

    remove_escape(input);

    printf("Processed command: %s\n", input);

    return 0;
}

```

```

balaji_p@Harshi:~/2520030104_OSSP$ nano escape.c
balaji_p@Harshi:~/2520030104_OSSP$ gcc escape.c -o escape
balaji_p@Harshi:~/2520030104_OSSP$ ./escape
Enter command: hello\ world
Processed command: hello world
balaji_p@Harshi:~/2520030104_OSSP$

```

7) Process execution

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    char command[100];

    printf("OSSP-Shell> ");

    scanf("%s", command);

    pid_t pid = fork();

    if(pid == 0)
    {
        execlp(command, command, NULL);

        printf("Command not found\n");

        exit(1);
    }

    else
    {
        wait(NULL);

        printf("Command completed\n");
    }

    return 0;
}

```

```

balaji_p@Harshi:~/2520030104_OSSP$ nano execute.c
balaji_p@Harshi:~/2520030104_OSSP$ gcc execute.c -o execute
balaji_p@Harshi:~/2520030104_OSSP$ ./execute
OSSP-Shell> ls
escape      execute    history    program1   quote      shell      tokenize.c
escape.c    execute.c  history.c  program1.c quote.c     shell.c    tokenize.c
Command completed
balaji_p@Harshi:~/2520030104_OSSP$

```

8) Path Search

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    char *path = getenv("PATH");

    printf("System PATH:\n");

    printf("%s\n", path);

    char *token = strtok(path, ":");

    while(token != NULL)
    {
        printf("Directory: %s\n", token);

        token = strtok(NULL, ":");
    }

    return 0;
}

```

```

balaji_p@Harshi:~/2520030104_OSSP$ nano path.c
balaji_p@Harshi:~/2520030104_OSSP$ gcc path.c -o path
balaji_p@Harshi:~/2520030104_OSSP$ ./path
System PATH:
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/
games:/usr/lib/wsl/lib:/mnt/c/Program Files/Common Files/Oracle/Java/javapath:/mnt/
c/Program Files (x86)/Common Files/Oracle/Java/java8path:/mnt/c/Program Files (x86)
/Common Files/Oracle/Java/javapath:/mnt/c/WINDOWS/system32:/mnt/c/WINDOWS:/mnt/c/WI
NDOWS/System32/Wbem:/mnt/c/WINDOWS/System32/WindowsPowerShell/v1.0:/mnt/c/WINDOWS/
System32/OpenSSH:/mnt/c/Program Files/Java/jdk-21/bin:/mnt/c/Program Files/nodejs/
:/mnt/c/Program Files/Git/cmd:/mnt/c/Program Files/MySQL/MySQL Shell 8.0/bin:/mnt/
c/Users/Balaji P/AppData/Local/Programs/Python/Launcher:/mnt/c/Users/Balaji P/AppD
ata/Local/Microsoft/WindowsApps:/mnt/c/Users/Balaji P/AppData/Local/Programs/Micros
oft VS Code/bin:/mnt/c/Users/Balaji P/AppData/Roaming/npm:/mnt/c/Users/Balaji P/App
Data/Local/Programs/Kiro/bin:/mnt/c/Users/Balaji P/AppData/Local/Programs/mongosh:/
mnt/c/Users/Balaji P/AppData/Local/Programs/Antigravity IDE/bin:/snap/bin
Directory: /usr/local/sbin
Directory: /usr/local/bin
Directory: /usr/sbin
Directory: /usr/bin
Directory: /sbin
Directory: /bin
Directory: /usr/games
Directory: /usr/local/games

```

9) Variable Expansion

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    char variable[50];

    printf("Enter variable name: ");

    scanf("%s", variable);

    char *value = getenv(variable);

    if(value)
    {
        printf("Value: %s\n", value);
    }

    else
    {
        printf("Variable not found\n");
    }

    return 0;
}

```

```

balaji_p@Harshi:~/2520030104_OSSP$ nano getenv.c
balaji_p@Harshi:~/2520030104_OSSP$ gcc getenv.c -o getenv
balaji_p@Harshi:~/2520030104_OSSP$ ./getenv
Enter variable name: PATH
Value: /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr
/local/games:/usr/lib/wsl/lib:/mnt/c/Program Files/Common Files/Oracle/Java/javapat
h:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/java8path:/mnt/c/Program File
s (x86)/Common Files/Oracle/Java/javapath:/mnt/c/WINDOWS/system32:/mnt/c/WINDOWS:/m
nt/c/WINDOWS/System32/Wbem:/mnt/c/WINDOWS/System32/WindowsPowerShell/v1.0:/mnt/c/W
INDOWS/System32/OpenSSH:/mnt/c/Program Files/Java/jdk-21/bin:/mnt/c/Program Files/
nodejs:/mnt/c/Program Files/Git/cmd:/mnt/c/Program Files/MySQL/MySQL Shell 8.0/bin
:/mnt/c/Users/Balaji P/AppData/Local/Programs/Python/Launcher:/mnt/c/Users/Balaji
P/AppData/Local/Microsoft/WindowsApps:/mnt/c/Users/Balaji P/AppData/Local/Programs
/Microsoft VS Code/bin:/mnt/c/Users/Balaji P/AppData/Roaming/npm:/mnt/c/Users/Balaj
i P/AppData/Local/Programs/Kiro/bin:/mnt/c/Users/Balaji P/AppData/Local/Programs/mo
ngosh:/mnt/c/Users/Balaji P/AppData/Local/Programs/Antigravity IDE/bin:/snap/bin
balaji_p@Harshi:~/2520030104_OSSP$

```

10) Buildin Commands

```

#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <stdlib.h>

int main()
{
    char command[100];

    while(1)
    {
        printf("OSSP-Shell> ");

        scanf("%s", command);

        if(strcmp(command, "exit") == 0)
        {
            break;
        }

        else if(strcmp(command, "pwd") == 0)
        {
            char cwd[200];

            getcwd(cwd, sizeof(cwd));

            printf("%s\n", cwd);
        }

        else if(strcmp(command, "cd") == 0)
        {
            char path[100];

            scanf("%s", path);

            if(chdir(path) != 0)
            {
                printf("Directory error\n");
            }
        }
    }
}

```

[Read 49 lines]

```

balaji_p@Harshi:~/2520030104_OSSP$ nano builtins.c
balaji_p@Harshi:~/2520030104_OSSP$ gcc builtins.c -o builtins
balaji_p@Harshi:~/2520030104_OSSP$ ./builtins
OSSP-Shell> pwd
/home/balaji_p/2520030104_OSSP
OSSP-Shell> cd
exit
Directory error
OSSP-Shell> exit
balaji_p@Harshi:~/2520030104_OSSP$ nano builtins.c
balaji_p@Harshi:~/2520030104_OSSP$

```

11)History Buffer

```

#include <stdio.h>
#include <string.h>

#define SIZE 10

char history[SIZE][100];

int count = 0;

void add_history(char *cmd)
{
    if(count < SIZE)
    {
        strcpy(history[count], cmd);
        count++;
    }
}

void show_history()
{
    int i;

    for(i = 0; i < count; i++)
    {
        printf("%d : %s\n", i + 1, history[i]);
    }
}

int main()
{
    char command[100];

    while(1)
    {
        printf("OSSP-Shell> ");

        fgets(command, 100, stdin);

        command[strlen(command) - 1] = '\0';

```

[Read 56 lines]

```

balaji_p@Harshi:~/2520030104_OSSP$ nano history_function.c
balaji_p@Harshi:~/2520030104_OSSP$ gcc history_function.c -o history_function
balaji_p@Harshi:~/2520030104_OSSP$ ./history_function
OSSP-Shell> ls
Executed : ls
OSSP-Shell> pwd
Executed : pwd
OSSP-Shell> hello
Executed : hello
OSSP-Shell> history
1 : ls
2 : pwd
3 : hello
OSSP-Shell> exit
balaji_p@Harshi:~/2520030104_OSSP$ nano history_function.c
balaji_p@Harshi:~/2520030104_OSSP$

```

12)Pipe

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];

    pipe(pipefd);

    pid_t pid = fork();

    if(pid == 0)
    {
        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[0]);

        execlp("ls", "ls", NULL);
    }

    else
    {
        wait(NULL);

        dup2(pipefd[0], STDIN_FILENO);

        close(pipefd[1]);

        execlp("grep", "grep", ".c", NULL);
    }

    return 0;
}

```

```

balaji_p@Harshi:~/2520030104_OSSP$ nano pipe.c
balaji_p@Harshi:~/2520030104_OSSP$ gcc pipe.c -o pipe
balaji_p@Harshi:~/2520030104_OSSP$ ./pipe
builtins.c
escape
escape.c
execute
execute.c
getenv.c
history.c
history_function
history_function.c
path.c
pipe.c
program1.c
quote.c
shell.c
tokenize.c
balaji_p@Harshi:~/2520030104_OSSP$

```

13) Output Redirection

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int fd;

    fd = open("output.txt",
              O_WRONLY | O_CREAT | O_TRUNC,
              0644);

    dup2(fd, STDOUT_FILENO);

    close(fd);

    printf("Output redirected successfully\n");

    return 0;
}
```

```
balaji_p@Harshi:~/2520030104_OSSP$ nano redirect.c
balaji_p@Harshi:~/2520030104_OSSP$ gcc redirect.c -o redirect
balaji_p@Harshi:~/2520030104_OSSP$ ./redirect
balaji_p@Harshi:~/2520030104_OSSP$ cat output.txt
Output redirected successfully
balaji_p@Harshi:~/2520030104_OSSP$
```

14)Advanced Redirection

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int fd;

    fd = open("log.txt",
              O_WRONLY | O_CREAT | O_APPEND,
              0644);

    dup2(fd, STDOUT_FILENO);

    close(fd);

    printf("New log entry\n");

    int errorfd;

    errorfd = open("error.txt",
                  O_WRONLY | O_CREAT | O_TRUNC,
                  0644);

    dup2(errorfd, STDERR_FILENO);

    close(errorfd);

    fprintf(stderr, "Sample error message\n");

    return 0;
}
```

```
balaji_p@Harshi:~/2520030104_OSSP$ nano logging.c
balaji_p@Harshi:~/2520030104_OSSP$ gcc logging.c -o logging
balaji_p@Harshi:~/2520030104_OSSP$ ./logging
balaji_p@Harshi:~/2520030104_OSSP$ cat log.txt
New log entry
balaji_p@Harshi:~/2520030104_OSSP$
```

15)Background jobs

```

#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int main()
{
    char command[50];

    printf("OSSP-Shell> ");

    scanf("%s", command);

    pid_t pid = fork();

    if(pid == 0)
    {
        printf("Running background process PID: %d\n",
            getpid());

        execlp(command, command, NULL);

        exit(1);
    }

    else
    {
        printf("Shell returned immediately\n");
    }

    return 0;
}

```

```

balaji_p@Harshi:~/2520030104_OSSP$ gcc background.c -o background
balaji_p@Harshi:~/2520030104_OSSP$ ./background
OSSP-Shell> ls
Shell returned immediately
Running background process PID: 1847
background    escape.c      history.c      output.txt    program1.c    shell.c
background.c  execute      history_function  path          quote         tokenize
builtins      execute.c    history_function.c  path.c        quote.c       tokenize.c
builtins.c    getenv       log.txt        pipe          redirect
error.txt     getenv.c    logging        pipe.c        redirect.c
escape        history      logging.c      program1      shell
balaji_p@Harshi:~/2520030104_OSSP$

```

16) Job Table

```

#include <stdio.h>

struct Job
{
    int id;
    int pid;
    char status[20];
};

int main()
{
    struct Job jobs[5];

    jobs[0].id = 1;

    jobs[0].pid = 1234;

    sprintf(jobs[0].status, "Running");

    printf("Job List\n");

    printf("ID PID STATUS\n");

    for(int i = 0; i < 1; i++)
    {
        printf("%d %d %s\n",
                jobs[i].id,
                jobs[i].pid,
                jobs[i].status);
    }

    return 0;
}

```

```

balaji_p@Harshi:~/2520030104_OSSP$ nano jobs.c
balaji_p@Harshi:~/2520030104_OSSP$ gcc jobs.c -o jobs
balaji_p@Harshi:~/2520030104_OSSP$ ./jobs
Job List
ID PID STATUS
1 1234 Running
balaji_p@Harshi:~/2520030104_OSSP$

```

17)Signal Basics

```

#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig)
{
    printf("\nSignal received: %d\n", sig);
}

int main()
{
    signal(SIGINT, handler);

    while(1)
    {
        printf("Running...\n");

        sleep(2);
    }

    return 0;
}

```

```

balaji_p@Harshi:~/2520030104_OSSP$ nano signal.c
balaji_p@Harshi:~/2520030104_OSSP$ gcc signal.c -o signal
balaji_p@Harshi:~/2520030104_OSSP$ ./signal
Running...
Running...
Running...
^C
Signal received: 2
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
^Z
[1]+  Stopped                  ./signal
balaji_p@Harshi:~/2520030104_OSSP$

```

18)SIGINT Handler

```

#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void sigint_handler(int sig)
{
    printf("\nCannot terminate shell\n");
}

int main()
{
    signal(SIGINT, sigint_handler);

    while(1)
    {
        printf("OSSP-Shell Running\n");

        sleep(2);
    }

    return 0;
}

```

```

balaji_p@Harshi:~/2520030104_OSSP$ nano shell_signal.c
balaji_p@Harshi:~/2520030104_OSSP$ gcc shell_signal.c -o shell_signal
balaji_p@Harshi:~/2520030104_OSSP$ ./shell_signal
OSSP-Shell Running
OSSP-Shell Running
OSSP-Shell Running
^C
Cannot terminate shell
OSSP-Shell Running
OSSP-Shell Running
^C
Cannot terminate shell
OSSP-Shell Running
OSSP-Shell Running
OSSP-Shell Running
OSSP-Shell Running
OSSP-Shell Running
^C
Cannot terminate shell
OSSP-Shell Running
OSSP-Shell Running
OSSP-Shell Running
OSSP-Shell Running
OSSP-Shell Running
^Z
[2]+  Stopped                  ./shell_signal
balaji_p@Harshi:~/2520030104_OSSP$

```

20) SIGTSTP Handler

```

#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void stop_handler(int sig)
{
    printf("\nProcess stopped signal: %d\n", sig);
}

int main()
{
    signal(SIGTSTP, stop_handler);

    while(1)
    {
        printf("Process active\n");

        sleep(2);
    }

    return 0;
}

balaji_p@Harshi:~/2520030104_OSSP$ nano sigtstp.c
balaji_p@Harshi:~/2520030104_OSSP$ gcc sigtstp.c -o sigtstp
balaji_p@Harshi:~/2520030104_OSSP$ ./sigtstp
Process active
Process active
^Z
Process stopped signal: 20
Process active
Process active
Process active
^Z
Process stopped signal: 20
Process active
Process active
Process active
Process active

```

21)Module Integration

```

#include <stdio.h>

void parser()
{
    printf("Parser module loaded\n");
}

void executor()
{
    printf("Executor module loaded\n");
}

void history()
{
    printf("History module loaded\n");
}

int main()
{
    printf("OSSP Shell Integration Test\n");

    parser();

    history();

    executor();

    printf("All modules connected successfully\n");

    return 0;
}

```

```

balaji_p@Harshi:~/2520030104_OSSP$ nano integration.c
balaji_p@Harshi:~/2520030104_OSSP$ gcc integration.c -o integration
balaji_p@Harshi:~/2520030104_OSSP$ ./integration
OSSP Shell Integration Test
Parser module loaded
History module loaded
Executor module loaded
All modules connected successfully
balaji_p@Harshi:~/2520030104_OSSP$

```

22)Valgrind Test

```

#include <stdio.h>
#include <stdlib.h>

int main()
{
    char *memory;

    memory = malloc(100);

    if(memory)
    {
        printf("Memory allocated\n");
    }

    free(memory);

    printf("Memory released\n");

    return 0;
}

```

```

balaji_p@Harshi:~/2520030104_OSSP$ nano memory.c
balaji_p@Harshi:~/2520030104_OSSP$ gcc memory.c -o memory
balaji_p@Harshi:~/2520030104_OSSP$ ./memory
Memory allocated
Memory released
balaji_p@Harshi:~/2520030104_OSSP$

```

23)Performance Test

```

#include <stdio.h>
#include <time.h>

int main()
{
    clock_t start, end;

    start = clock();

    for(int i = 0; i < 1000000; i++)
    {
    }

    end = clock();

    double time_taken;

    time_taken = (double)(end - start) / CLOCKS_PER_SEC;

    printf("Execution time: %lf seconds\n", time_taken);

    return 0;
}

```

```

balaji_p@Harshi:~/2520030104_OSSP$ nano time.c
balaji_p@Harshi:~/2520030104_OSSP$ gcc time.c -o time
balaji_p@Harshi:~/2520030104_OSSP$ ./time
Execution time: 0.002494 seconds
balaji_p@Harshi:~/2520030104_OSSP$ nano time.c
balaji_p@Harshi:~/2520030104_OSSP$

```

24)Complete Shell

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    char command[100];

    while(1)
    {
        printf("OSSP-Shell> ");

        fgets(command, 100, stdin);

        command[strlen(command) - 1] = '\0';

        if(strcmp(command, "exit") == 0)
        {
            printf("Shell closed\n");
            break;
        }

        pid_t pid = fork();

        if(pid == 0)
        {
            execlp(command, command, NULL);

            printf("Command not found\n");

            exit(1);
        }

        else
        {
            wait(NULL);
        }
    }
}

```

[Read 43 lines]

```
balaji_p@Harshi:~/2520030104_OSSP$ nano complete_shell.c
balaji_p@Harshi:~/2520030104_OSSP$ gcc complete_shell.c -o complete_shell
balaji_p@Harshi:~/2520030104_OSSP$ ./complete_shell
OSSP-Shell> pwd
/home/balaji_p/2520030104_OSSP
OSSP-Shell> ls
background      execute.c       jobs.c         pipe.c         shell_signal.c
background.c    getenv         log.txt       program1      signal
builtins       getenv.c      logging      program1.c    signal.c
builtins.c     history       logging.c    quote        sigtstp
complete_shell history.c      memory       quote.c       sigtstp.c
complete_shell.c history_function memory.c     redirect     time
error.txt      history_function.c output.txt   redirect.c   time.c
escape        integration   path        shell        tokenize
escape.c      integration.c path.c      shell.c      tokenize.c
execute       jobs         pipe        shell_signal
OSSP-Shell> exit
Shell closed
```